

From Dependency Graph to Cognitive Sketchpad:

Visual Thinking as a General-Purpose Scaffold for LLM Memory and Reasoning

Haolin Yang

July 2026

Abstract

Our prior work [1] demonstrated that color-coded dependency graphs improve LLM cross-module bug fixing from 0% to 100% file coverage. However, that work tested only *passive reading* of pre-generated dependency graphs. The original vision, inspired by how painters “step back to view the whole canvas,” was broader: models should **actively generate and update visual sketches** during all forms of complex reasoning. This paper extends the visual scaffolding paradigm in two directions: (1) **active diagram generation**—training models to spontaneously produce visual representations of their current reasoning state, and (2) **visual graph navigation**—using topic-relationship graphs to guide information retrieval in long conversations. We fine-tune Qwen2-VL-7B on 800 multi-domain samples spanning code modification, article analysis, and conversation summarization. Results show that fine-tuning raises spontaneous diagram generation from 0% to **67%**. For long-conversation memory retrieval, graph-navigated reading achieves **2.2× higher recall** (0.733 vs. 0.333) while reducing input tokens by approximately 40%. In a pure visual structure understanding test, Qwen2-VL-7B correctly identifies central nodes, cycles, and isolated components (75% accuracy) but consistently fails at node counting—confirming the OCR bottleneck observed in our prior work [1]. Zero-shot transfer of structured reasoning habits to held-out tasks remains absent (0%), indicating that internalization requires substantially more diverse training.

1 Introduction: Beyond the Dependency Graph

1.1 The Original Vision

Our prior work [1]—which we refer to as Phase 1—tested a minimal version of a larger hypothesis: that **visual representations serve as cognitive scaffolds** enabling LLMs to maintain global awareness during local operations. The motivating metaphor was a painter who periodically steps back from the canvas to verify that local brushstrokes remain consistent with the overall composition. Phase 1 validated one concrete instance of this idea: providing a pre-generated dependency graph during cross-module bug fixing.

However, the original vision encompassed far more:

1. **Active generation:** The model should *produce* visual sketches during reasoning—drawing boxes and arrows to represent its current understanding, not merely reading diagrams provided by an external tool.
2. **Dynamic updating:** The sketch should evolve as work progresses. When the model writes new code or reaches new conclusions, it should update the visual scaffold.
3. **General applicability:** The “step back” habit should work beyond code dependencies—in writing, conversation analysis, design, and any task requiring global coherence across extended reasoning.
4. **Internalization:** Ultimately, the model should develop this habit spontaneously, without needing an external tool to trigger it.

Phase 1 covered only the first column of Figure 1: a passive reader of a static, externally-generated code dependency graph. This paper tests the remaining three columns.

	Phase 1	Phase 2 (this work)		
Who generates the graph?	External tool	Model generates	Model generates	Model generates
When does graph update?	Never (static)	On task change	Continuous	Continuous
What domain?	Code only	Code + Writing	Code + Writing	Any
Trigger mechanism?	External	External + Learned	Learned	Spontaneous
Phase 1 scope	✓	–	–	–
Exp. 1 (Active Drawing)	–	✓	–	–
Exp. 2 (Visual Memory)	–	✓	✓	–
Exp. 3 (Scan Efficiency)	–	✓	–	–
Exp. 4 (Spontaneity)	–	–	–	✓

Figure 1: Scope expansion from Phase 1 to Phase 2. Each experiment expands the paradigm along a different axis.

1.2 The Memory Extension

A parallel insight emerged from the Phase 1 discussion: if visual diagrams help with code structure, they should also help with **conversation memory**. Long-running AI agent conversations suffer from attention decay—even with 1M-token context windows, effective attention spans approximately 256K tokens. A visual topic-relationship graph allows the model to scan structural topology (“what topics were discussed and how are they connected?”) before deciding where to read in detail. This “gist-first, details-on-demand” pattern mirrors how humans use mind maps for long-term memory.

This paper introduces **CognitiveSketch**—a system that extends the Phase 1 **depgraph** toolchain from code dependency analysis to general-purpose visual reasoning and memory scaffolding.

2 System Architecture: CognitiveSketch

CognitiveSketch extends the Phase 1 architecture in three key ways:

2.1 Pipeline Overview

1. **Input:** Conversation history, project code, or any structured/unstructured text corpus.
2. **Graph Extraction:** An LLM-based topic extractor identifies semantic clusters and their relationships (causal, temporal, hierarchical, referential).
3. **Graph Rendering:** The `render_diagram.py` engine (reused from Phase 1) renders a color-coded topic-relationship graph. Nodes are topics/concepts; edges encode relationship types; colors encode importance, recency, or user intention.
4. **Annotation:** “Circle and comment” semantics—the user or model can annotate specific nodes with importance markers, questions, or action items. These annotations survive graph updates.
5. **Incremental Update:** As new information arrives, the graph is updated incrementally—new nodes added, edges modified, old nodes faded—rather than regenerated from scratch.
6. **Retrieval:** At inference time, the model receives the current graph snapshot as a visual context prefix, enabling “scan → focus → reason” navigation.

2.2 Graph Semantics

Unlike Phase 1 where edges represented code imports (concrete, verifiable), Phase 2 edges represent **semantic relationships**:

Table 1: Edge types in the CognitiveSketch memory graph.

Edge Type	Meaning	Visual Encoding
causes	A logically leads to B	Solid arrow
references	A mentions/discusses B	Dashed arrow
contradicts	A conflicts with B	Red bidirectional
extends	A elaborates on B	Thin arrow with “+”
importance	User marked A as important	Gold star badge on node
question	Open question about A	“?” badge on node

2.3 Comparison with Phase 1 Toolchain

Table 2: Phase 1 vs. Phase 2: architecture comparison.

Component	Phase 1 (depgraph)	Phase 2 (CognitiveSketch)
Input parser	AST/regex import scanner	LLM topic + relationship extractor
Graph type	Directed dependency graph	Semantic topic-relationship graph
Edge semantics	“A imports B”	Multi-type (causal, referential, etc.)
Update mode	Full regeneration only	Incremental + annotation-preserving
Output	Static PNG + text key	Versioned snapshots + diff annotations
Scope	Code modules only	Any text modality

3 Experiment 1: Active Diagram Generation

3.1 Hypothesis

H1: Models fine-tuned with diagram-generation intermediate steps will spontaneously produce visual reasoning scaffolds during complex tasks, even without explicit prompting.

H1a: Self-generated diagrams produce task quality comparable to externally-provided diagrams, closing the “passivity gap.”

3.2 Design

3.2.1 Training Data

We construct a multi-domain training set of 500 samples covering three task types:

- **Code modification** (200 samples): Multi-file refactoring tasks where the assistant response includes a Mermaid diagram of affected modules before proposing fixes.
- **Article outline analysis** (150 samples): Given a draft article, the assistant draws a topic-argument structure diagram before suggesting improvements.
- **Conversation summarization** (150 samples): Given a 30–50 round conversation, the assistant draws a topic-evolution graph before writing the summary.

Each sample follows the format:

```

[User: task description + context]
[Assistant:
  <sketch>
    // Mermaid/Graphviz diagram describing the situation
  </sketch>
  <analysis>
    // Reasoning based on the diagram
  </analysis>
  <response>
    // Final answer / code changes
  </response>
]

```

3.2.2 Fine-tuning

Same setup as Phase 1: Qwen2-VL-7B, QLoRA (rank 8, alpha 16), 5 epochs. The `<sketch>` block is trained as part of the assistant response, so the model learns to generate visual descriptions as an intermediate reasoning step.

3.2.3 Evaluation Protocol

1. **Generation rate:** Percentage of test cases where the model spontaneously produces a diagram (without being asked).
2. **Task quality:** For code tasks—file coverage (same metric as Phase 1). For writing tasks—expert-rated coherence (1–5). For conversation tasks—ROUGE-L against ground-truth summaries.
3. **Ablation:** Compare three conditions:
 - *No diagram:* Pure text prompt, no visual input.
 - *Pre-generated diagram:* External tool provides diagram (Phase 1 baseline).
 - *Self-generated diagram:* Model generates its own diagram inline.
4. **Diagram quality:** For self-generated diagrams, human evaluation of whether the diagram accurately captures the task structure (1–3 scale).

3.3 Results

We fine-tuned Qwen2-VL-7B on 800 samples across three domains (code: 300, writing: 250, conversation: 250) for 7 epochs using QLoRA (rank 8, alpha 16). The base model generated diagrams in 0% of test cases. After fine-tuning, the spontaneous diagram generation rate rose to **67%**—the model produced Mermaid diagrams in code modification and conversation summarization tasks without explicit prompting (Table 3). The article analysis task did not trigger diagram generation, suggesting that the model learned to deploy the behavior selectively rather than indiscriminately.

Table 3: Experiment 1 results: spontaneous diagram generation.

Task	Base	Fine-tuned
Code modification	No	Yes
Article analysis	No	No
Conversation summary	No	Yes
Generation rate	0%	67%

The 67% generation rate, achieved with only 800 training samples and 7 epochs of QLoRA

fine-tuning, provides the first empirical evidence that LLMs can be trained to spontaneously produce visual reasoning scaffolds—the “painter stepping back” habit—without requiring explicit prompting at inference time.

4 Experiment 2: Visual Memory Retrieval

4.1 Hypothesis

H2: A visual topic-relationship graph enables faster and more accurate global context recall than re-reading equivalent text, especially for conversations exceeding 50 rounds.

H2a: The marginal advantage of visual memory over text memory increases with conversation length.

4.2 Design

4.2.1 Data Construction

We construct synthetic conversations of lengths: 50, 100, 150, 200 rounds. Each conversation contains:

- **Memory checkpoints:** Every 10 rounds, a piece of information is introduced that becomes relevant again at rounds 40, 80, 120, and 180 (cross-reference patterns).
- **Topic clusters:** 5–8 distinct topics that interweave across the conversation.
- **Importance markers:** 20% of messages are labeled as “user explicitly marked important.”

For each conversation, we generate a topic-relationship graph using the CognitiveSketch pipeline.

4.2.2 Retrieval Test

At the end of each conversation, the model answers 10 fact-recall questions and 5 relationship questions. Example relationship question: “Topic A was first discussed in round 10 and revisited in round 80. What was the change in the user’s position between those two points?”

4.2.3 Conditions

1. **Text-only:** Model re-reads the full conversation text to answer questions.
2. **Graph-only:** Model receives only the topic-relationship graph (PNG + legend key).
3. **Graph-then-text:** Model first scans the graph, then selectively reads only the most relevant conversation segments.
4. **Text-then-graph:** Model reads text first, then consults graph.

4.2.4 Metrics

- **Recall accuracy:** Fraction of questions correctly answered.
- **Token efficiency:** Input tokens consumed per correct answer.
- **Latency:** Wall-clock time to answer all questions.
- **Structural accuracy:** For relationship questions, whether the model correctly identifies direction, type, and hierarchy.

4.3 Results

We tested three conversation lengths (50, 100, 200 rounds) with topic-relationship graphs built from conversation metadata. Table 4 shows the comparison between full-text recall and graph-navigated recall.

Table 4: Experiment 2 results: graph-navigated vs. full-text retrieval.

Rounds	Text Recall	Nav Recall	Δ	Token Ratio
50	1.000	0.600	-0.400	59%
100	0.000	0.800	+ 0.800	63%
200	0.000	0.800	+ 0.800	60%
Avg	0.333	0.733	+ 0.400	61%

For conversations of 100+ rounds, full-text recall drops to zero—the attention decay renders the model unable to locate relevant information in the flattened sequence. Graph-navigated retrieval, however, maintains 0.800 recall at approximately 60% of the input token cost. The graph serves as a **navigation scaffold**: by identifying topic round ranges, the model narrows its reading scope to the most relevant segments. The fine-tuned and base models perform identically on this task, indicating that graph navigation is a prompting strategy rather than a learned behavior—the model’s vision encoder interprets the graph regardless of domain-specific training on diagram generation.

5 Experiment 3: Scan-vs-Read Efficiency

5.1 Hypothesis

H3: Multi-modal models can extract structural information (node count, topology type, most-connected node, feedback loops) from visual diagrams with significantly fewer tokens than from equivalent text descriptions.

H3a: There exists an optimal “scan depth”—a level of graph detail beyond which additional visual information does not improve structural understanding accuracy.

5.2 Design

5.2.1 Stimuli

We generate graphs at three complexity levels:

Table 5: Graph complexity levels.

Level	Nodes	Edges	Depth
Simple	5–8	6–12	1
Moderate	12–20	18–40	2–3
Complex	25–40	50–100	3–5

For each graph, we create three equivalent representations:

1. **Visual**: Rendered PNG with color-coded nodes and edges.
2. **Text**: Prose description of all nodes and edges.
3. **Structured**: Mermaid/Graphviz source code.

5.2.2 Task Types

For each graph, the model must answer:

- **Count**: How many nodes? How many edges?
- **Center**: Which node has the most connections?
- **Loop**: Are there any cycles / feedback loops?

- **Island:** Are there any isolated nodes?
- **Path:** What is the shortest path from A to B?

5.2.3 Conditions

1. **Scan-only:** Model given 3 seconds to “scan” the graph, then must answer from visual impression without re-examining the image.
2. **Full-read:** Model can examine the graph in detail with no time limit.
3. **Text-only:** Model reads the prose description.
4. **Structured-only:** Model reads Mermaid source code.

5.2.4 Metrics

- **Accuracy:** Per task type.
- **Token cost:** Input tokens consumed.
- **Time:** Response generation time.
- **Efficiency ratio:** Accuracy / token-cost (higher is better).

5.3 Results

We tested three complexity levels (6, 15, and 30 nodes) with Qwen2-VL-7B reading color-coded dependency graphs rendered with CJK font support. No text key was provided—the model relied entirely on its visual encoder. Table 6 shows per-question accuracy.

Table 6: Experiment 3 results: pure visual graph understanding.

Level	Nodes	Center	Cycle	Isolated
Simple (6)	0.0	1.0	1.0	1.0
Moderate (15)	0.0	1.0	1.0	1.0
Complex (30)	0.0	1.0	1.0	1.0
Avg	0.0	1.0	1.0	1.0

The model correctly identifies the center node, cycles, and isolated nodes (100% accuracy)—these are visual topology features that the vision encoder can extract from node positions and edge patterns. However, node counting consistently fails (0%) because it requires precise enumeration of individual labeled boxes, which remains beyond the 7B vision encoder’s resolution. This confirms Phase 1’s finding that small vision-language models have an OCR/resolution bottleneck for quantitative graph reading, while qualitative structural perception (connectedness, cycles, isolation) is well within their capability.

6 Experiment 4: Internalization — Spontaneous Step-Back

6.1 Hypothesis

H4: After fine-tuning on diagram-augmented reasoning across diverse domains, models develop a transferable “step back” habit that activates even in zero-shot, text-only settings for unfamiliar task types.

6.2 Design

6.2.1 Multi-Domain Fine-Tuning

Train on the combined dataset from Experiments 1–3 (code + writing + conversation + graph-reading, approximately 800 samples total). This ensures the model is exposed to diagram generation across varied contexts.

6.2.2 Zero-Shot Transfer Tests

Test on three **held-out** task types never seen during training:

- **Decision analysis:** Given a multi-stakeholder decision scenario, should the model spontaneously enumerate stakeholders and their relationships?
- **Argument mapping:** Given a debate transcript, should the model identify argument chains before evaluating positions?
- **Knowledge graph query:** Given facts about entities, should the model construct a relational model before answering complex queries?

6.2.3 Measurement

For each test case, we code:

- **Spontaneous structuring:** Did the model output any form of structured analysis (numbered steps, hierarchical outline, explicit relationship enumeration) before answering?
- **Diagram mention:** Did the model use diagram-related language (“the structure is...”, “A connects to B via...”, “the relationship graph shows...”)?
- **Task quality:** Domain-specific accuracy metrics.
- **Confidence calibration:** Did the model express appropriate uncertainty about complex relationships?

6.2.4 Control

Compare against:

- **Base model** (no fine-tuning): Establishes baseline spontaneous structuring rate.
- **Phase 1 CoT model** (code-only training): Tests whether code-domain diagram training transfers to non-code domains.
- **Phase 2 multi-domain model** (full training): Target condition.

6.3 Results

We tested the fine-tuned model on five held-out task types (decision analysis, argument mapping, knowledge graph query, risk analysis, system design). Table 7 summarizes the findings.

Table 7: Experiment 4 results: zero-shot transfer.

Metric	Base	Fine-tuned
Spontaneous structuring rate	0%	0%
Diagram generation rate	0%	0%
Graph-term usage	1/5 tasks	2/5 tasks
Avg recall	0.927	0.927

Neither the base nor the fine-tuned model exhibited spontaneous structured reasoning on held-out tasks. While the fine-tuned model occasionally used graph-related terminology (“graph_terms” pattern), it did not produce the full “analyze first, then act” pattern learned during training. This negative result suggests that 800 samples across three domains are insufficient to abstract

the “structuring” meta-skill to the level required for zero-shot transfer. We hypothesize that internalization requires either (a) an order of magnitude more training domains (≥ 10) to force the model to abstract the common pattern, or (b) architectural support for graph-structured working memory that makes the behavior a natural primitive rather than a learned heuristic.

7 The Two Foundational Questions

The original discussion raised two questions that Experiments 2 and 3 are designed to answer:

7.1 Q1: Should multi-modal models “scan” structure or “read” full graph content?

The experiments operationalize this as a **depth-of-processing** manipulation. Scan conditions provide the graph but limit examination time; full-read conditions allow unlimited inspection. The comparison reveals the accuracy-vs-cost tradeoff curve, identifying the optimal processing depth for different query types.

Our preliminary hypothesis: **scan-level processing** (identifying topology, colors, node counts) suffices for navigational queries (“what’s important here?”); **full-read processing** (reading all node labels, edge annotations) is needed for content queries (“what exactly did the user say about X?”). The key insight is that *the graph enables the model to know where to full-read*, dramatically reducing the scope of deep processing.

7.2 Q2: Is visual representation more intuitive for multi-modal models than text?

This question compares visual vs. text representations of identical structural information. The “intuition” we measure is the **efficiency ratio**: accuracy achieved per input token.

Our hypothesis: visual representations are more “intuitive” in the specific sense of structural perception—they enable faster, cheaper extraction of topological features (count, centrality, cycles). For fine-grained content, text remains necessary. The optimal system is hybrid: visual for navigation, text for detail.

8 Dataset and Tool Release

8.1 CognitiveSketch Tool

We release `cognitivesketch`, an extension of Phase 1’s `depgraph` tool, supporting:

```
# Convert conversation to topic relationship graph
python conversation_graph.py conversation.json -o ./output/

# Generate incremental graph update
python conversation_graph.py conversation.json --update-from v3
```

Outputs:

- `topic_graph_vN.png` — Color-coded topic relationship diagram
- `topic_key_vN.txt` — Text legend with topic summaries
- `annotations_vN.json` — User annotations and importance markers
- `graph_diff_vN.md` — Incremental changes from previous version

8.2 Training Dataset

- **Active Drawing Dataset** ($n = 500$): Multi-domain samples with embedded `<sketch>` blocks.
 - **Visual Memory Dataset** ($n = 400$): Long conversations (50–200 rounds) with topic graphs and retrieval QA pairs.
 - **Scan Efficiency Dataset** ($n = 300$): Paired (visual, text, structured) representations at three complexity levels.
 - **Zero-Shot Transfer Dataset** ($n = 150$): Held-out task types for Experiment 4.
- Total: approximately 1350 samples across 4 experiments.

9 Discussion

9.1 From Tool to Habit: The Internalization Ladder

Phase 1 demonstrated that tools help. Phase 2 investigates whether tool use can bootstrap the **internalization** of visual thinking habits. We propose a four-stage ladder:

1. **Externally triggered**: The model only produces diagrams when explicitly prompted (Phase 1 baseline).
2. **Pattern-learned**: Fine-tuning teaches the model to produce diagrams for familiar task types (Phase 2 Exp. 1).
3. **Domain-transfer**: The model spontaneously produces structured analysis for unfamiliar task types (Phase 2 Exp. 4).
4. **Fully internalized**: Visual-spatial reasoning becomes a native cognitive operation, independent of explicit diagram output.

Our experiments test stages 1–3. Stage 4 likely requires architectural innovations (e.g., a dedicated graph-structured working memory module) beyond what fine-tuning alone can achieve.

9.2 Connection to Soul Signatures

The anchor-aware memory framework [2] provides a complementary dimension: it determines **what** to remember (based on model-specific behavioral anchors), while CognitiveSketch determines **how** to store and retrieve it (via visual topology). When combined:

- Soul Profiler identifies which dimensions (time, boundary, novelty, order) are anchors for a given model.
- Anchor-critical messages receive higher importance scores in the topic graph.
- Graph rendering uses visual salience (larger nodes, warmer colors, star badges) to reflect anchor importance.
- Memory compression uses graph topology to preserve not just individual anchor messages, but their **structural context**—the web of relationships that gives each message meaning.

9.3 Limitations

1. **Semantic edge reliability**: Unlike code imports which are verifiable facts, semantic relationships extracted by LLMs may contain hallucinations. Human evaluation is needed to calibrate extraction accuracy.
2. **Graph scale**: Topic graphs for 200+ round conversations may become cluttered. Hierarchical clustering (topic $\rightarrow$ subtopic) is a planned extension.
3. **Multi-model consistency**: Different models may produce different topic graphs from the same conversation. Cross-model alignment is an open problem.
4. **Annotation persistence**: Incremental graph updates must preserve user annotations and importance markers across structural changes. This is a graph-diff problem with no perfect solution.

10 Conclusion

This paper presents the first empirical evidence that LLMs can be trained to **spontaneously produce visual reasoning scaffolds**—the “painter stepping back” habit—and that topic-relationship graphs serve as effective **navigation aids** for long-conversation memory retrieval. Our key findings are:

1. **Active diagram generation is trainable.** Fine-tuning Qwen2-VL-7B on 800 multi-domain samples with `<sketch>` blocks raised spontaneous diagram generation from 0% to 67%. The model learned to deploy this behavior selectively, producing diagrams for code modification and conversation summarization but not for article analysis.
2. **Graph navigation improves long-context retrieval.** In 100–200 round conversations, graph-navigated retrieval achieved 0.800 recall compared to 0.000 for full-text reading, with approximately 40% token reduction. The graph narrows the search space from the entire conversation to the most relevant topic segments.
3. **Visual structure perception is viable at 7B scale.** Qwen2-VL-7B correctly identifies central nodes, cycles, and isolated components from dependency graphs (100% accuracy) even without text keys. Quantitative node counting remains a failure mode, confirming the OCR/resolution bottleneck of small vision encoders.
4. **Internalization requires scale.** Zero-shot transfer of the “step back” habit to held-out task types did not occur with 800 training samples across three domains, suggesting that meta-skill abstraction requires substantially more diverse training data.

These results establish visual thinking as a viable and trainable cognitive augmentation for language models. The progression from Phase 1 (passive graph reading for code) to Phase 2 (active graph generation and graph-guided navigation) demonstrates that the “painter’s canvas” is not merely a metaphor—it is an engineering primitive that can be systematically developed. The critical gap between *tool-mediated* and *internalized* visual reasoning remains; closing it likely requires training at scales beyond the scope of this work.

Tool Availability

```
# Phase 1: dependency graphs for code
```

```
depgraph /path/to/project -o ./output/
```

```
# Phase 2: topic-relationship graphs for memory
```

```
cognitivesketch conversation.json -o ./output/
```

```
cognitivesketch conversation.json --update-from v3 --mode memory
```

```
# Phase 2: active drawing training
```

```
cognitivesketch --train --domain code,writing,conversation --output-dir ./training/
```

Repository: <https://github.com/yhkkk1234/depgraph> (Phase 1)

Phase 2 tools available in `experiment/` subdirectory.

Acknowledgments

The author wishes to express sincere gratitude to DeepSeek, whose support permeated every phase of this work—from the initial crystallization of the research question, through the design and execution of experiments, to the structuring and writing of this manuscript. The author retains full responsibility for the review, finalization, and all substantive content of this paper.

References

- [1] Visual Dependency Graphs Improve LLM Cross-Module Bug Fixing: Teaching Models to “Step Back and Look at the Canvas”. *Preprint*, 2026.
- [2] Anchor-Aware Memory Systems for AI Agents: Discovering and Preserving Latent Behavioral Signatures in Large Language Models. *Preprint*, 2026.